

Introduction to Data Structure

UNIT - 1

DATA MANAGEMENT CONCEPT

- Till now, Program is used to generate output correctly.
- *But, with correct output program should run efficiently.*
- So, In order to write efficient program need to apply Data Management Concept.

What is Data?

- “Data is collection of RAW values which can be interrelated or not”

What is Information?

- “Information is collection of logically related Data which are pre-processed and interrelated.”
- So, Data Management System works on Data to convert them into Information.

DATA STRUCTURE

- Task of Data Management System
 - Data Collection
 - Organize those data in specific structure
 - Maintaining the routines for quality assurance
- Data Structure is crucial part of Data management System.

DATA STRUCTURE

- “Data Structure is a logical way to organize collection of interrelated and preprocessed data such that insertion, updating and deletion of those data can be efficient”

DATA STRUCTURE

- Data Structure is used in every field of Software Engineering like;
 - Compiler Design
 - DBMS
 - Artificial Intelligence
 - Numerical & Statistical Analysis
 - Simulation
 - Graphics
 - Operating System.
- To achieve efficiency, following three steps help us
 - Analysis of problem to determine I/P and O/P operations
 - Quantify resources constraints for each operation
 - Select the data structure that best meets requirements.

WHY DATA STRUCTURE?

- Learning data structures is essential because writing code that works is only half the job in computer science—the other half is writing code that scales.
- As data grows from tens of records to millions, how that data is arranged in memory directly determines whether an application runs seamlessly or crashes under load.
- Core Reasons -
 - Optimization of Time and Space Complexity
 - Without the right structure: Finding an item in an unsorted list of \$1,000,000\$ elements requires scanning every single item one by one (Linear Time, $O(n)$).
 - With the right structure: Storing that same data in a balanced Binary Search Tree or Hash Table allows you to find the item almost instantaneously in logarithmic or constant time ($O(\log n)$ or $O(1)$).

WHY DATA STRUCTURE?

- Efficient Memory Management
 - Data structures govern how memory is allocated and released in hardware.
 - Arrays require continuous, fixed memory blocks, which makes index lookups instant but makes resizing expensive.
 - Linked Lists allocate memory dynamically on demand across non-contiguous locations, preventing wasted memory when data sizes fluctuate unpredictably.
- Modeling Complex Real-World Relationships
 - Abstract real-world concepts rarely map directly to primitive values like numbers or strings. Data structures supply the mathematical blueprints to model complex domains:
 - Graphs represent interconnected networks like social connections, electrical grids, or road navigation systems.
 - Trees represent hierarchical relationships like file directories, HTML DOM structures, or organizational charts.
 - Stacks model nested state management, enabling features like browser history ("Back" button) or text editor undo/redo functionality.

WHY DATA STRUCTURE?

- Foundation for Algorithm Design
 - Algorithms and data structures are intrinsically linked—an algorithm is the step-by-step procedure, while the data structure is the vessel that holds the state. High-level techniques like dynamic programming, graph traversal, and sorting algorithms rely heavily on specific data structures (like priority queues or matrices) to execute correctly.

TYPES OF DATA STRUCTURE

- Basically Two types;

1. Primitive Data Structure

- “Primitive Data Structure is defined from Basic or Primary Data Types”
- Primitive data structure can be store only single value at a time
- It supports basic types like int, float, double, char, void etc...

2. Non-Primitive Data Structure

- “Non-Primitive DS is defined from derived data types”
- It can store collection of values of same type or different types.
- For ex. Array, Queue, Stack, Linked list, Graph , Tree
- It can be divided into TWO sub types.

TYPES OF DATA STRUCTURE

i. Linear Data Structure

- “Structure that can store values in sequence in Memory that is known as Linear DS”
- For Ex: Array, Stack, Queue, Link List

ii. Non-Linear Data Structure

- “A structure which does not store values in sequence inside memory is known as Non-Linear DS”
- For ex; Graph, Tree

DIFFERENT DATA STRUCTURE

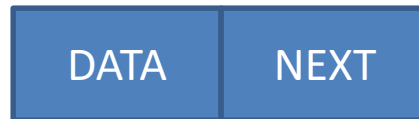
1. ARRAY

- It's Linear Data Structure.
- “Array is sequential collection of Data elements of same data type”
- Each data elements are arranged sequentially in memory location
- Each elements are referred by it's index only
- Declaration of Array likes - `DataType var_name [size];`
- For ex; - `int number[10];`
- Disadvantages
 - It has fixed size
 - Array needs sequential memory locations which may not always be possible
 - Adding and removing data elements are complex due to shifting operation of each elements.

DIFFERENT DATA STRUCTURE

2. Linked List

- It's Linear Data Structure.
- Whatever problem of contiguous memory location problem is faced in array, that can be solved in Linked List.
- It is dynamic and flexible data structure
- Because we can easily add and delete elements from the list
- Each element is composition of mainly two parts, first is DATA and second is NEXT (address of next element)



NODE

- Each elements are allocated space and added to list without size constraints.
- Last element of list contains NULL as address in “NEXT” part.
- Search operation is very slow.
- It requires more memory space due to value and address to be stored.

DIFFERENT DATA STRUCTURE

3. Stack

- It's Linear Data Structure.
- Using Array or Linked List, we can implement this Data Structure
- “Stack is collection of elements in sequence in which last inserted element is retrieved first’
- This approach is also known as “Last In First Out” (LIFO)
- To maintain track of each element inserted Last, “TOP” variable is used, which stores “index” of Last inserted element.
- If TOP is -1 than stack is empty.
- It has Three operation
 1. PUSH : to insert value in stack at TOP
 2. POP : to retrieve value from TOP
 3. PEEP : to return value at TOP position

DIFFERENT DATA STRUCTURE

4. Queue

- It's Linear Data Structure.
- Using Array or Linked List, we can implement this Data Structure
- “Queue is sequential collection of data elements in which first inserted element is retrieved first from the list”
- This approach is known as “First In First Out” (FIFO)
- To maintain track of each element, ‘FIRST’ and ‘REAR’ trackers are used.
- ‘FIRST’ tracks the element which was inserted first in Queue
- ‘REAR’ tracks the element which was inserted last in Queue
- At insertion operation, ‘REAR’ is getting updated and at deletion operation ‘FIRST’ is getting updated.
- If Queue is empty, then FIRST and REAR both are -1
- Three Operations: Insert, Delete and Display
- Two Conditions: Overflow (when queue is full) and Underflow (when queue is empty)

DIFFERENT DATA STRUCTURE

5. Graph

- “A Graph is collection of Vertices and Edges that connect all vertices.”
- It's Non-Linear Data Structure.
- Used to implement Mathematical concept
- Every node in Graph is connected to every other node.
- Two nodes which are connected by common edge is known as Neighbor.
- Searching, Shortest Path, Spanning Tree, etc... are operations can be performed on Graph.

6. Tree

- “Tree is a collection of elements in which each node contains left pointer, right pointer and data elements.”
- Tree can be General Tree, Binary Tree, Forest, etc...
- It's Non-Linear Data Structure.
- It contains ROOT node as first node in tree, if ROOT is null then tree is empty
- ROOT points to it's two sub-tree – Left Sub-tree and Right Sub-tree
- It provides QUICK search, insert operation but SLOW delete operation.

ALGORITHMS

- An Algorithm is a blue print to write a Program which can be solve a problem.
- “An Algorithm is collection of ordered instruction set that lead us to solution of problem”
- Algorithm is considered as Effective Pseudo Code
- There can be multiple algorithm available for single problem, so the choice of algorithm depends on TIME and MEMORY availability.
- **FEATURES of ALGORITHM**
 1. **Finiteness** : An algorithm must have finite number of steps.
 2. **Effectiveness** : An algorithm must have process that's effectiveness on input to generate correct output.
 3. **Definite** : An algorithm must give surety to generate exact output as per out requirements.
 4. **Input/Output** : Each algorithm must have some i/p and at least one o/p.

ALGORITHMS

- Each algorithm must be efficient and correct to generate output as per our desire
- Efficiency is depending on two factors: TIME and SPACE.
- To measure these two factors, we need to do Analysis of Algorithm
- To analyze algorithm means to determine amount of Resource (Time and Space) needed to execute it.
- This amount of resource is considered as Complexity.
- Two types of Complexity

1. Time Complexity

- It indicates running time or execution time of program
- It depends mainly on size of input that is given to program
- Time complexity has three cases:
 1. Best case (Less time taken to execute)
 2. Worst case (More time taken to execute)
 3. Average case (Average time taken to execute)

ALGORITHMS

2. Space Complexity

- It indicates amount of computer memory required to store local or temporary variables' values.
- It also depends on size of input
- Two types
 - **Fixed Part:** Space needed to store instruction, constraint, variables etc...
 - **Variable Part:** Space needed to store recursion stack, dynamically created variables.